

Labo 16 – Verslag JavaScript deel 4

opleidingsonderdeel **Web Development I**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Tibo Dewanckele

docent: **Pim Debaere**

academiejaar **2025-2026**

Inhoud

Vragen	2
Opdracht: invulformulier.....	3
Opdracht: typeof	7
Opdracht: spaties op console	8
Opdracht: spaties met functie	9
Opdracht: Man van An	9

Vragen

Waarom is het potentieel een probleem als je GET gebruikt in een situatie waar het POST had moeten zijn?

Als je GET gebruikt in een situatie waar POST had moeten gebruikt geweest zijn dan zal er gevoelige informatie in de URL zichtbaar zijn.

Gebruik een <label> element om de betekenis van een bijbehorend input element te duiden. Als je een <label> gebruikt in combinatie met een <checkbox> of <radiobutton> zul je in deze geen tekst meer stoppen natuurlijk.

Waarom niet?

Omdat je de tekst al in label steekt en anders dubbel zou hebben.

Opdracht: invulformulier

Eindresultaat van het formulier

Persoonlijke informatie	
Voornaam*:	Your first name
Achternaam*:	Your last name
Sport	
Welke sport beoefen je?	
☐ Lopen	
☐ Zwemmen	
☐ Roeien	
Communicatievoorkeur	
Hoe kunnen we je bereiken?	
☐ Per post	
☐ E-mail	
Favorieten	
Favoriete film:	
Favoriete quote:	Schrijf je favoriete quote
Favoriete eten:	Hamburger
Favoriete browser:	
Beveiliging & contact	
Wachtwoord*:	
E-mailadres:	Geef je e-mailadres in
Favoriete site:	Geef de URL van je favoriete
Extra informatie	
Favoriete getal tussen 1 en 10:	
Favoriete getal tussen 11 en 1000:	
Wanneer ben je geboren?	dd/mm/yyyy
Belgische postcode*:	9999
Varia	
Zoeken naar iets:	
Wat is je lievelingskleur?	
Bestand kiezen Geen bestand gekozen	
Submit	

In de HTML starten we met **<form method="post">** in de body

Daarin komen al deze delen dat we kunnen zien op bovenstaande afbeelding.

Persoonlijke informatie

Persoonlijke informatie	
Voornaam*:	Your first name
Achternaam*:	Your last name

```
<fieldset>
  <legend>Persoonlijke informatie</legend>
  <label for="voornaam">Voornaam*:</label>
  <input type="text" name="voornaam" id="voornaam" placeholder="Your first name"
    required autofocus>
  <br>
  <label for="achternaam">Achternaam*:</label>
  <input type="text" name="achternaam" id="achternaam" placeholder="Your last name"
    required>
</fieldset>
```

- Hier gebruiken we een **fieldset** om te zorgen dat de **<legend>** / titel in het kader staat.
- Dan hebben we 2 inputvelden (**<input>**) met elk een **<label>** die via de **id** wordt gekoppeld.
- Als de website laadt dan wil ik dat de muis automatisch eerst op het inputveld van de voornaam staat dus geven we dit **autofocus**.
- Omdat we deze velden willen verplichten om in te vullen gebruiken we **required**.
- Via **placeholder** wordt er een tekst in het inputveld gezet totdat je er iets begint in te typen.

Sport

Sport

Welke sport beoefen je?

☐ Lopen

☐ Zwemmen

☐ Roeien

```
<fieldset>
  <legend>Sport</legend>
  <p>Welke sport beoefen je?</p>
  <input type="checkbox" name="sport" id="lopen" value="Lopen">
  <label for="lopen">Lopen</label><br>
  <input type="checkbox" name="sport" id="zwemmen" value="Zwemmen">
  <label for="zwemmen">Zwemmen</label><br>
  <input type="checkbox" name="sport" id="roeien" value="Roeien">
  <label for="roeien">Roeien</label><br>
</fieldset>
```

Hier maken we gebruik van **checkbox** als inputvelden, ook hier gebruiken we een **label**
→ we geven ze dezelfde **name** zodat ze een soort van gekoppeld zijn

Communicatievoorkeur

Communicatievoorkeur

Hoe kunnen we je bereiken?

☐ Per post

☐ E-mail

```
<fieldset>
  <legend>Communicatievoorkeur</legend>
  <p>Hoe kunnen we je bereiken?</p>
  <label for="post"></label>
  <input type="radio" name="communicatie" id="post" value="Per post">
  Per post<br>
  <label for="mail"></label>
  <input type="radio" name="communicatie" id="mail" value="Mail">
  E-mail<br>
</fieldset>
```

Hier maken we gebruik van **radio** als inputvelden, ook hier gebruiken we een **label**
→ de **name** moet hier hetzelfde zijn want anders kan je meerdere selecteren en zijn ze niet gekoppeld. Het is de bedoeling dat je maar 1 kan selecteren.

Favorieten

```

<fieldset>
  <legend>Favorieten</legend>
  <label for="fav_film">Favoriete film:</label>
  <input type="text" name="fav_film" id="fav_film">
  <br>
  <label for="fav_quote">Favoriete quote: </label>
  <input type="text" name="fav_quote" id="fav_quote"
    placeholder="Schrijf je favoriete quote">
  <br><br><br>
  <label for="fav_food">Favoriete eten:</label>
  <select id="fav_food" name="fav_food">
    <option value="Hamburger">Hamburger</option>
    <option value="Hot-dog">Hot-dog</option>
    <option value="Pizza">Pizza</option>
  </select><br>
  <label for="fav_browser">Favoriete browser:</label>
  <input type="text" name="fav_browser" id="fav_browser">
</fieldset>

```

Hier willen we een lijst van opties geven voor favoriete eten

→ hiervoor gebruiken we **<select>** en opnieuw heeft dit een **label** die gekoppeld is via **id**

→ via **<option>** voegen we iets toe aan de keuzelijst.

Beveiliging & Contact

Beveiliging & contact

Wachtwoord*:

E-mailadres:

Favoriete site:

```

<fieldset>
  <legend>Beveiliging & contact</legend>
  <label for="password">Wachtwoord*:</label>
  <input type="password" name="password" id="password"
    required><br>
  <label for="email">E-mailadres:</label>
  <input type="email" name="email" id="email"
    placeholder="Geef je e-mailadres in"><br>
  <label for="url">Favoriete site:</label>
  <input type="url" name="url" id="url"
    placeholder="Geef de URL van je favoriete site"><br>
</fieldset>

```

- Voor een wachtwoord kan je als type bij **input password** gebruiken, dan zal deze bolletjes tonen in plaats van het wachtwoord zelf.

- Voor een email gebruiken we type **email**, deze zal dan controleren of er iets dat lijkt op een e-mailadres werd ingegeven.

Extra informatie

Extra informatie

Favoriete getal tussen 1 en 10:

Favoriete getal tussen 11 en 1000:

Wanneer ben je geboren?



Belgische postcode*:

```
<fieldset>
  <legend>Extra informatie</legend>
  <label for="klein_getal">Favoriete getal tussen 1 en 10:</label>
  <input type="number" name="klein_getal" id="klein_getal" min="1" max="10">
  <br>
  <label for="groot_getal">Favoriete getal tussen 11 en 1000:</label>
  <input type="range" class="slider" id="groot_getal" value="11" min="11" max="1000">
  <br>
  <label for="geboortedatum">Wanneer ben je geboren?</label>
  <input type="date" name="geboortedatum" id="geboortedatum">
  <br>
  <label for="postcode">Belgische postcode*:</label>
  <input type="text" name="postcode" id="postcode" placeholder="9999"
    required pattern="^([1-8][0-9]{3}|9[0-8][0-9]{2}|99[0-8][0-9]|999[0-2])$" />
</fieldset>
```

- Om een **getal** in te voeren gebruiken we **number**
 - om dan tussen getallen kunnen we een **min** en een **max** meegeven
- Om een **slider** te maken gebruiken we **range**
 - hierbij geven we een standaard **value**, een **min** en een **max**
- Om een **datum** te kunnen invoeren gebruiken we **date**
- We kunnen beperken wat er wordt ingegeven via **pattern**
 - `="^(...)$"`
 - **[bereik]** bevat de waarden op die positie in dat getal
 - **{waarde}** is het aantal keer dat hij de [] voor zich moet herhalen in het getal
 - Via | kan je extra opties toevoegen

Varia

Varia

Zoeken naar iets:

Wat is je lievelingskleur?

Bestand kiezen

Geen bestand gekozen

Submit

```
<fieldset>
  <legend>Varia</legend>
  <label for="search">Zoeken naar iets:</label>
  <input type="text" name="search" id="search"><br>
  <label for="color">Wat is je lievelingskleur?</label>
  <input type="color" name="color" id="color"><br>
  <label for="file"></label>
  <input type="file" id="file">
</fieldset>
<button type="submit">Submit</button>
```

- Via type **color** kan je een kleur laten kiezen
- Via type **file** kan je een bestand uit je verkenner laten kiezen.
- Om een knop te maken gebruiken we **<button type="">**
 - bij type moet je submit, reset of niks zetten

Opdracht: typeof

```
let leeftijd : number = 34;

let intrest : number = 0.12;

let isGevaarlijk : boolean = true;

let vandaag : Date = new Date(); const print = (message) : void => {
  console.log(message);
}

console.log(typeof(leeftijd));
console.log(typeof(intrest));
console.log(typeof(isGevaarlijk));
console.log(typeof(vandaag));
```

Dit is het resultaat

number	script.js:10
number	script.js:11
boolean	script.js:12
object	script.js:13

Opdracht: spaties op console

Resultaat



We hebben een inputveld en een knop toegevoegd in de HTML

```
<label for="text">Inputveld: </label><input type="tekst" id="text">  
<button id="spatiebutton">Zet spaties ertussen</button>
```

Om te zorgen dat er tussen elk teken 1 spatie komt heb ik volgende javascript gebruikt.

```
const setup = () => {  
  let button = document.getElementById("spatiebutton");  
  
  button.addEventListener("click", spatieZetten);  
}
```

We halen eerst de knop op via **id** en dan voegen we een eventlistener toe om te weten wanneer hij geklikt wordt, als erop geklikt word dan zal hij onderstaande methode uitvoeren.

```
const spatieZetten = () => {  
  let text = document.getElementById("text");  
  
  console.log(text.value.replaceAll(" ", "").split("").join(" "));  
}  
window.addEventListener("load", setup);
```

We halen het inputveld op met **id** tekst

→ dan willen we op dat zijn **value** verschillende methode toepassen

→ Eerst willen we alle spaties weg zodat bij splitsen er maar 1 spatie tussen tekens is: **replaceAll()**

→ Dan zullen we alle tekens splitsen via **split()**

→ Dan maken we er een string van gescheiden door spaties via **join()**

Opdracht: spaties met functie

Dit heeft hetzelfde resultaat als vorige opdracht maar is met een parameter doorgegeven via de functie.

```
const setup = () => {
  let button = document.getElementById("spatiebutton");
  let input = document.getElementById("text");

  button.addEventListener("click", () => {maakMetSpaties(input.value)})
}
```

We roepen hier binnen de accolades de methode met een **parameter** op
→ de parameter is de input dat we hebben opgehaald

```
const maakMetSpaties = (inputText) => {
  let result = inputText.replaceAll(" ", "").split("").join(" ");
  console.log(result);
}
```

Nu moeten we het hier niet meer ophalen maar gebruiken we de parameter.

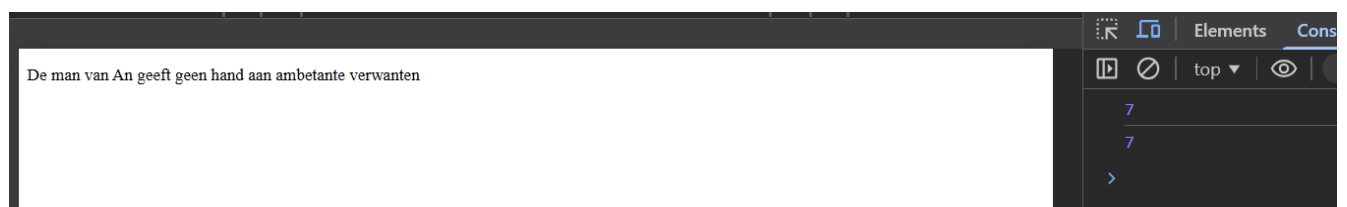
Opdracht: Man van An

In de HTML zetten we een p tag met een zin die we in de javascript zullen ophalen.

```
<p id="tekst">De man van An geeft geen hand aan ambetante verwanten</p>
```

In het script hebben we 2 methodes om te tellen hoe vaak an voorkomt in de zin hierboven. 1 die gebruik maakt van **indexOf()** en 1 die gebruik maakt van **lastIndexOf()**

```
const setup = () => {
  aantalAnTellenMetIndexOf();
  aantalAnTellenMetLastIndexOf();
}
```



IndexOf

We halen eerst de zin op en zetten deze lowerCase want we willen de hoofdletters negeren. Dan houden we de index bij om te weten waar we nog in de zin aan het zoeken zijn, dan hebben we nog een variabele aantal om bij te houden hoeveel “an” we al zijn tegengekomen.

```
const aantalAnTellenMetIndexOf = () => {  
  let tekst = document.getElementById("tekst").textContent.toLowerCase();  
  
  let index = 0;  
  let aantal = 0;  
  while (tekst.indexOf("an", index) !== -1) {  
    index = tekst.indexOf("an", index) + 2;  
    aantal++;  
  }  
  console.log(aantal);  
}
```

Als **indexOf()** niks vind dan krijg je **-1 terug**, hierop wordt gecontroleerd

De index is de startpositie dat hij begint met zoeken in de tekst.

We updaten deze iedere keer zodat hij na de gevonden an begint te zoeken.

We doen nog eens +2 want de index dat hij teruggeeft is bij de de “a” en we willen na an starten.

lastIndexOf()

```
const aantalAnTellenMetLastIndexOf = () => {  
  let tekst = document.getElementById("tekst").textContent.toLowerCase();  
  
  let index = tekst.length;  
  let aantal = 0;  
  
  while (tekst.lastIndexOf("an", index) !== -1) {  
    index = tekst.lastIndexOf("an", index) - 2;  
    aantal++;  
  }  
  console.log(aantal);  
}
```

Nu moeten we vanachter starten, dus stellen we de index gelijk aan de lengte van de tekst. Nu moeten we dus ook -2 doen als we een “an” vinden voor de index.